

Lucrare de laborator : 6

Interoperabilitatea COM

1. Introducere

Componentele COM au o arhitectură internă diferită de componentele .NET, prin urmare nu sunt compatibile în mod inerent. Cele mai multe organizații, care și-au construit aplicațiile pentru întreprinderi pe obiecte COM pentru serviciile lor de nivel mediu, nu pot renunța la investițiile pe aceste soluții. Aceste componente vechi ar trebui să fie exploatate prin codul gestionat în cadrul .NET. Acesta este locul unde intervine interoperabilitatea; este un program RCW (Runtime Callable Wrapper) care traduce apeluri specifice de la clienți gestionați în cereri de invocare specifice COM pe componente COM nesupravegheate. Metoda de apel pe RCW va face componentele NET să creadă că vorbesc doar cu o altă componentă .NET.

COM reprezintă Component Object Model, care este specificația binară pentru reutilizarea codului software. Acesta impune un standard pentru interfețele prin care codul client discută la clasele de componente. Deoarece ASP clasic a fost limitată în funcționalitatea sa, pentru a îndeplini anumite sarcini într-o aplicație Web ASP, a fost necesară o componentă COM. De exemplu, crearea de imagini, examinarea jurnalului de evenimente sau efectuarea unei solicitări HTTP la distanță erau toate sarcinile care necesitau o componentă COM.

În plus, componentele COM au fost folosite în mod obișnuit pentru a încapsula logica de afaceri, după cum sa discutat în Utilizarea Business Objects în aplicația Web ASP. Un site web eCommerce ar putea crea o componentă COM care să aibă metode cum ar fi `CalculateShippingCosts (shoppingCartID)` și `CalculateArrivalDate (shipToZipCode)`, care ar calcula costurile de expediere pentru articolele dintr-un coș de cumpărături dintr-un anumit utilizator și va calcula cât timp va dura un transfer pentru a ajunge la cumpărător. Avantajul de a introduce astfel de reguli de afaceri și logică în componentele COM include o lizibilitate și o mentenabilitate îmbunătățită a codului. Deoarece paginile Web ASP.NET au acces la clasele din .NET Framework, paginile Web ASP.NET pot, fără componente, să execute activități cum ar fi crearea de imagini, examinarea Jurnalului de evenimente sau efectuarea unor cereri de HTTP la distanță. Cu toate acestea, este încă o idee înțeleaptă de a încapsula logica de afaceri în componente, așa cum este discutat în articolul Afișarea claselor personalizate într-un DataGrid. Componentele utilizate de o pagină Web ASP.NET sunt în mod obișnuit denumite componente .NET și diferă de componentele COM în mai multe moduri importante. Aceste diferențe înseamnă că paginile Web ASP.NET nu pot folosi nativ componentele COM. Mai degrabă, trebuie făcute unele împiedicări pentru a permite interoperabilitatea între componentele clasice COM și o pagină Web ASP.NET. În acest articol vom examina conceptele și vom prezenta un exemplu care demonstrează interoperabilitatea COM în .NET Framework.

COM reprezintă Component Object Model, care este o specificație binară pentru reutilizarea codului software. Acesta impune un standard pentru interfețele prin care codul client discută la clasele de componente. Interfața `IUnknown` a componentei ajută la menținerea unui număr de referință al numărului de clienți care utilizează componenta. Când acest număr scade la zero, componenta este descărcată. Toate componentele ar trebui să implementeze interfața `IUnknown`. Numărul de referință este menținut prin metodele `IUnknown::AddRef()` și `IUnknown::Release()`, iar descoperirea interfeței este gestionată prin `IUnknown::QueryInterface()`.

2. Furnizarea interoperabilității

Componentele COM au o arhitectură internă diferită de componentele .NET, prin urmare nu sunt compatibile în mod inerent. Cele mai multe organizații, care și-au construit aplicațiile pentru întreprinderi pe obiecte COM pentru serviciile lor de nivel mediu, nu pot renunța la investițiile pe aceste soluții. Adică, pentru ca astfel de companii să migreze la ASP.NET, trebuie să existe o modalitate prin care noile pagini Web ASP.NET să utilizeze componente vechi, vechi, COM.

Pentru a avea o componentă COM folosită pe o pagină Web ASP.NET, vom folosi ceea ce se numește un RCW (Runtime Callable Wrapper). RCW traduce apeluri specifice de pe pagina Web ASP.NET în cereri de invocare specifice COM pe o componentă COM. Când folosim RCW-uri, pagina Web ASP.NET va crede că vorbește doar cu o altă componentă .NET în loc să vorbească cu o componentă COM.

Următorul grafic descrie rolul RCW. În partea stângă este aplicația .NET (vă puteți gândi la aceasta ca pe pagina noastră Web ASP.NET). În partea dreaptă este componenta COM moștenită pe care o dorim ca pagina noastră Web ASP.NET să funcționeze. În mijloc se află RCW, care acceptă cererile primite de pe pagina Web ASP.NET, le traduce astfel încât acestea să poată fi manipulate de componenta COM și apoi să le treacă pe componenta COM. În esență, RCW acționează ca proxy gestionat pentru componenta COM neordonată.

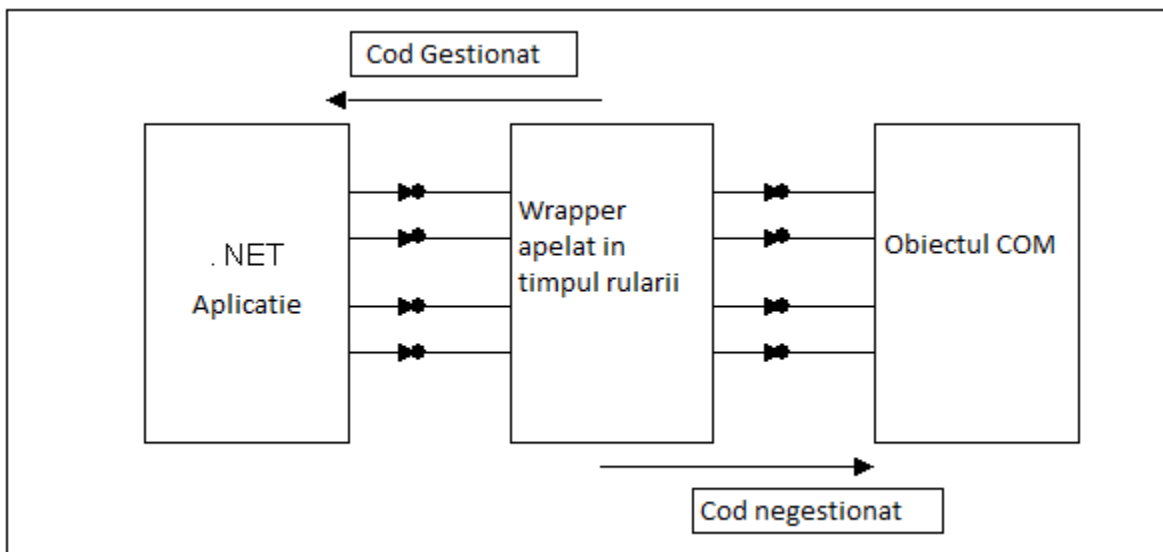


Figura 6.1: COM

Când facem un apel de metodă, acesta merge pe wrapper și nu pe obiectul propriu-zis. Wrapperul gestionează evoluția pe durata de viață a componentei COM.

3. Cum este legarea componentelor în interoperabilitate?

Legarea se referă la informații despre metode, proprietăți, evenimente etc. pe care un client trebuie să le cunoască despre obiect. Încă mai putem folosi tehnica veche bună a legăturilor în interoperabilitatea .NET, și anume legăturile timpurii și târzii.

Legarea anterioară: clienții obțin informații despre tipul de timp de compilare din biblioteca de componente a componentei.

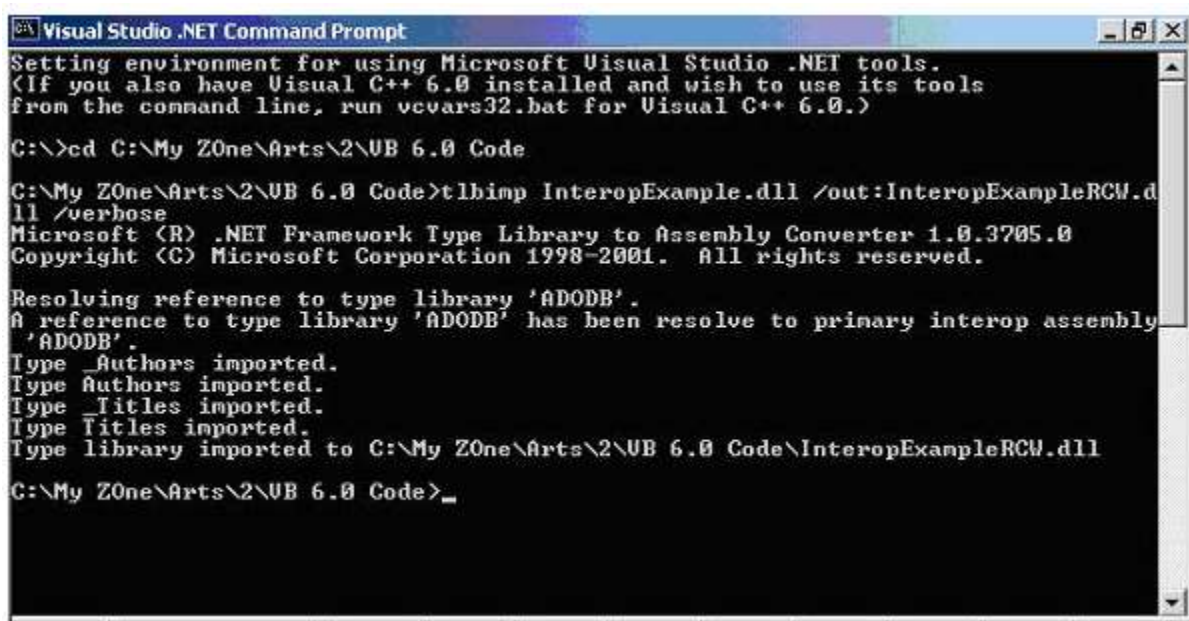
Angajarea târzie: Clienții nu primesc informații de tip bogat ale obiectului, este cunoscut în timpul rulării.

4. Cum implementăm interoperabilitatea COM?

Există două moduri de a genera un wrapper de metadata de gestionat:

- Utilizarea utilitarului Importator de tip bibliotecă.
- VS.NET IDE

Importatorul de tip de bibliotecă (tlbimp.exe) este o sintaxă a liniei de comandă, care convertește definiția de tip specifică COM într-o bibliotecă de tip COM în definiții echivalente pentru un ansamblu de .NET wrapper. Implicit, utilitarul oferă ansamblului de împachetare același nume ca DLL COM.



```
Visual Studio .NET Command Prompt
Setting environment for using Microsoft Visual Studio .NET tools.
<If you also have Visual C++ 6.0 installed and wish to use its tools
from the command line, run vcvars32.bat for Visual C++ 6.0.>

C:\>cd C:\My ZOne\Arts\2\VB 6.0 Code

C:\My ZOne\Arts\2\VB 6.0 Code>tlbimp InteropExample.dll /out:InteropExampleRCW.d
ll /verhose
Microsoft (R) .NET Framework Type Library to Assembly Converter 1.0.3705.0
Copyright (C) Microsoft Corporation 1998-2001. All rights reserved.

Resolving reference to type library 'ADODB'.
A reference to type library 'ADODB' has been resolve to primary interop assembly
'ADODB'.
Type _Authors imported.
Type Authors imported.
Type _Titles imported.
Type Titles imported.
Type library imported to C:\My ZOne\Arts\2\VB 6.0 Code\InteropExampleRCW.dll

C:\My ZOne\Arts\2\VB 6.0 Code>_
```

Figura 6.2 Command prompt VS

Exemplul de mai sus generează un ansamblu de metadata cu numele "InteropExampleRCW.dll" din componenta COM "InteropExample.dll" folosind următoarea sintaxă la promptul de comandă VS.NET:

Notă: rezolvă intern referințele ADODB în COM. Prin argumentul "out", putem specifica numele de asamblare dorit.

Importatorul de biblioteci de tip interoghează biblioteca de tip COM DLL și traduce informațiile în format .NET. Ansamblul de metadate astfel generat conține clase de împachetare care pot fi utilizate în orice client .NET, de exemplu, clienți C # Windows. RCW este creat în zbor ori de câte ori este creată o componentă și acționează ca tipuri gestionate pe tipuri specifice de date COM.

VS.NET IDE ne ajută, de asemenea, să generăm un set de metadate: Faceți clic pe Project -> Add reference -> tab COM

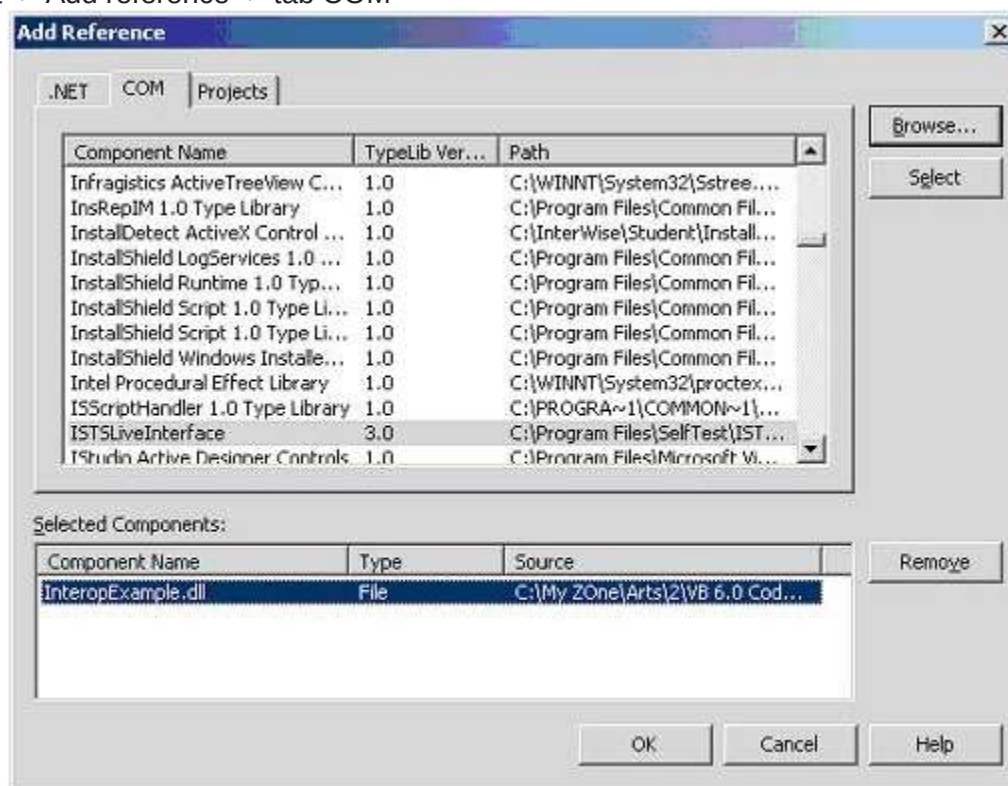


Figura 6.3 Project COM..

Fila listează componentele înregistrate pe mașina locală, selectează COM DLL dorit și adaugă la lista componentelor selectate. VS.NET generează automat un ansamblu de metadate punând clasele furnizate de acea componentă într-un spațiu de nume cu același nume ca DLL-ul COM.

5. Cum este structura unui ansamblu Wrapper?

Pentru fiecare clasă importată într-un ansamblu de învelitori, se generează două clase de înfășurare. Informațiile specifice COM pot fi vizualizate utilizând utilitarul MSIL Disassembler (ildasm.exe) la promptul de comandă VS.NET:

```
ildasm InteropExampleRCW.dll
```

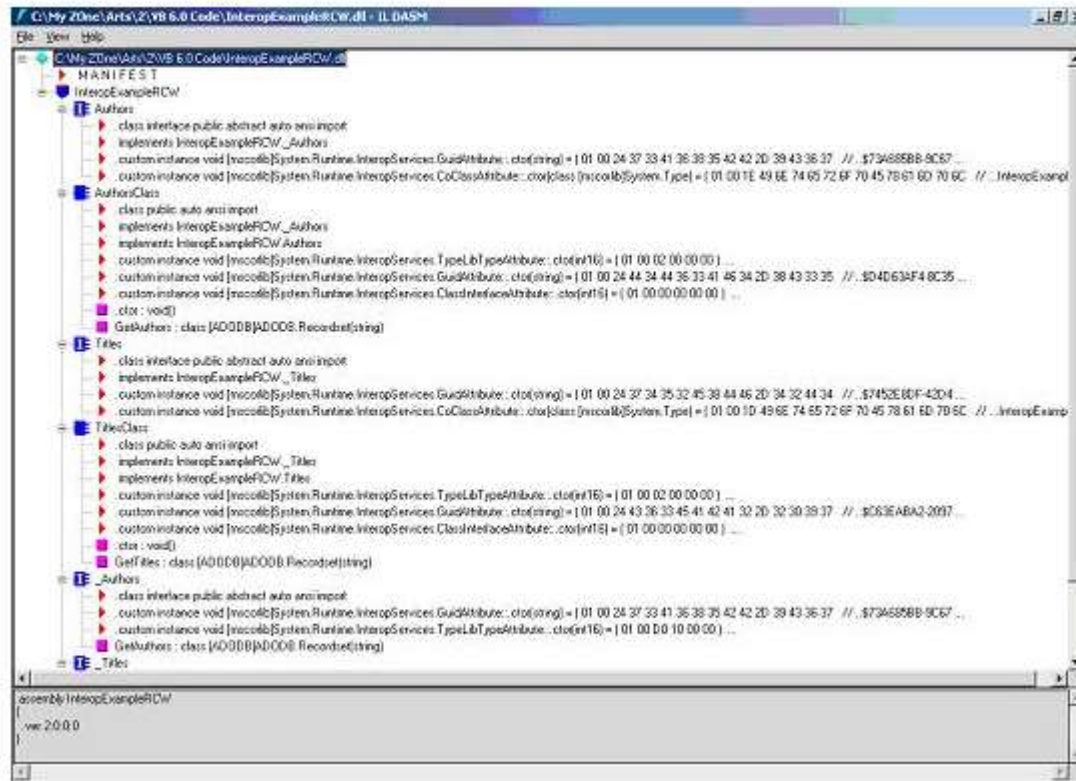


Figura 6.4 Ansamblul Wrapper.

InteropExample.dll (dezvoltat în versiunea VB 6.0) are următoarele clase publice: Autori titlurile Ansamblul generat are patru clase:

Autori
AuthorsClass
Titlurile
TitlesClass

Prima este interfața având același GUID ca și clasa COM originală, a doua este o clasă concretă a cărei instanță urmează a fi creată. Clasa de beton este sufixată cu cuvântul "Clasă". Clasa concreta implementează toate interfețele care sunt suportate de clasa COM originală. Toate tipurile generate sunt plasate sub spațiul de nume unic InteropExampleRCW.

Se referențiază DLL-ul de asamblare a metadatelor în proiect și se utilizează metodele și proprietățile sale:

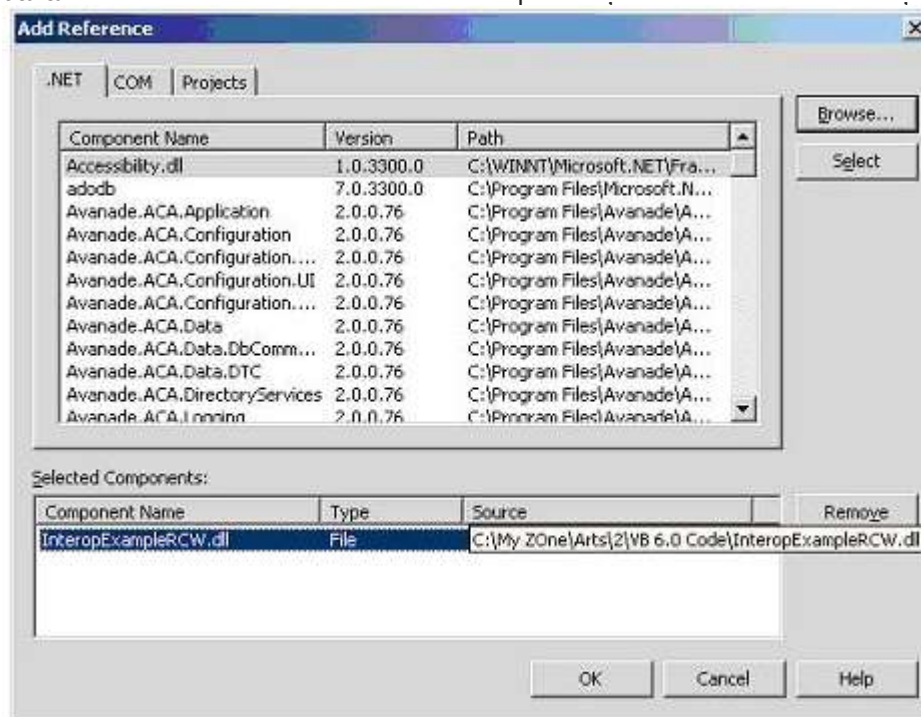


Figura 6.5: Referențiere Metadata.

Creați un nou proiect de aplicație C # Windows. Glisați următoarele formate din Label, TextBox și DataGridView și denumiți-le în consecință; formularul va apărea după cum urmează:

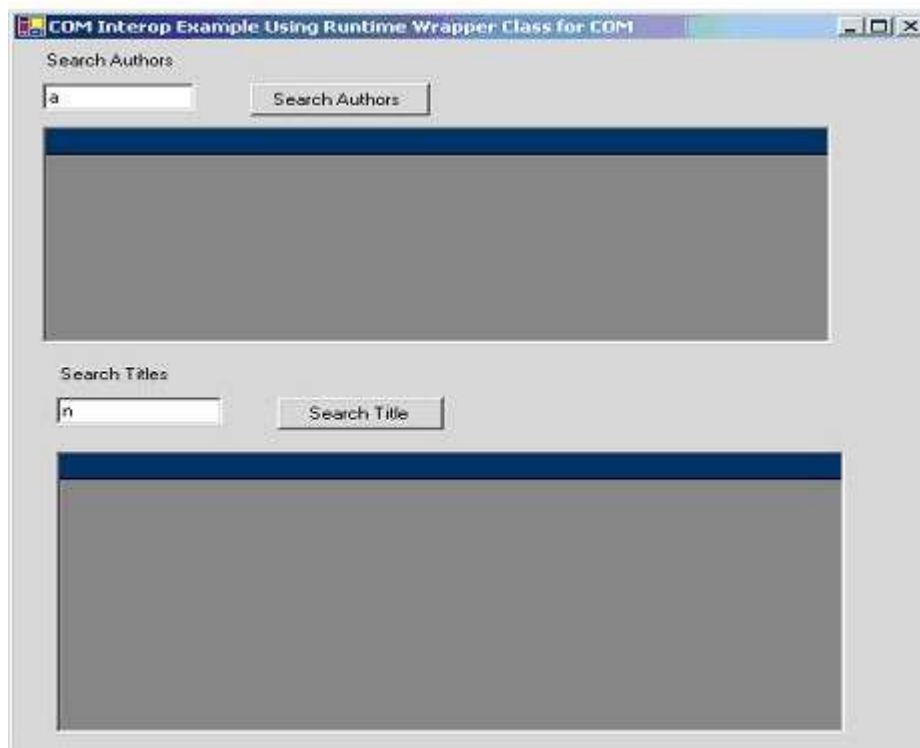


Figura 6.6: Proiect nou c#.

Importați ansamblurile necesare:

```
using System.Data;
using System.Data.OleDb;
```

Pentru a utiliza căutarea autorilor din clasa Autori, aveți un codul detaliat mai jos pentru evenimentul cu click pe butonul "căutării autorului":

```
private void btnSearchAutori_Click(object sender, System.EventArgs e)
{
    //create an instance of AutoriClass from wrapper assembly
    InteropExampleRCW.AutoriClass myAuthorRCW =
        new InteropExampleRCW.AutoriClass();

    DataSet dsAuthorList=new DataSet("Autori");

    OleDbDataAdapter daAuthRecs=new OleDbDataAdapter();
    ADODB.Recordset rsAutori=new ADODB.Recordset();

    rsAutori=myAuthorRCW.GetAutori(txtAutori.Text.ToString());

    //invoke method from the RCW
    daAuthRecs.Fill(dsAuthorList,rsAutori,"Autori");

    dataGridAutori.SetDataBinding(dsAuthorList,"Autori");
}
```

Pentru a utiliza Căutarea titlurilor din clasa Titlu, codul de mai jos este pentru evenimentul de pe butonul Căutare titlu:

```
private void btnSearchTitlu_Click(object sender, System.EventArgs e)
{
    InteropExampleRCW.TitluriClass myTitluriRCW =
        new InteropExampleRCW.TitluriClass();

    DataSet dsTitluList=new DataSet("Titluri");

    OleDbDataAdapter daTitluRecs=new OleDbDataAdapter();

    ADODB.Recordset rsTitluri=new ADODB.Recordset();

    rsTitluri=myTitluriRCW.GetTitluri(txtTitlu.Text.ToString());

    daTitluRecs.Fill(dsTitluList,rsTitluri,"Titluri");

    dataGridTitlu.SetDataBinding(dsTitluList,"Titluri");
}
```

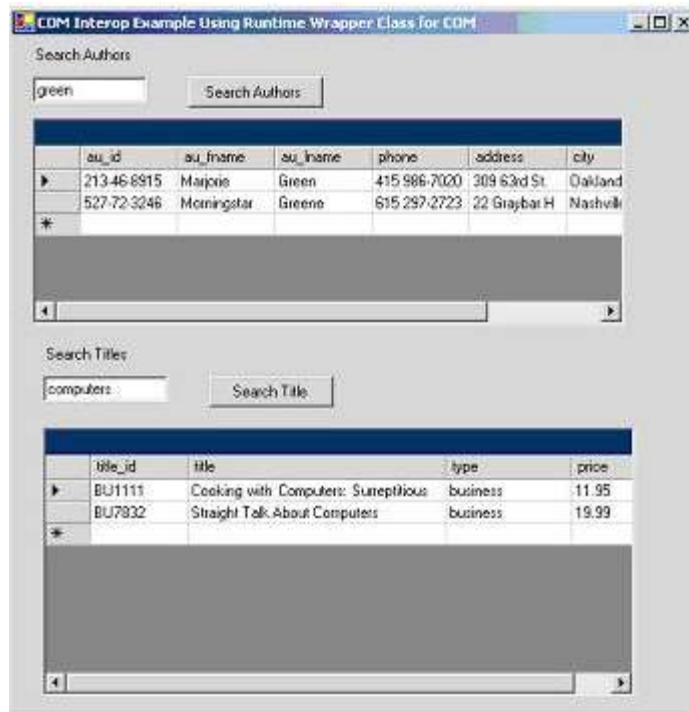



Figura 6.7: Interfata program final

Cum pot elibera obiecte COM?

Runtime Callable Wrapper este gestionată în sine; prin urmare, durata sa de viață este controlată de Runtime Common Language. Componenta COM este eliberată din memorie atunci când colectorul de gunoi apelează metoda `Finalize ()` pe RCW. Pe plan intern, RCW solicită eliberarea () a interfeței cunoscute IU pe obiectul COM.

Pentru a elimina explicit obiectele COM din memorie, invoca metoda statică pe clasa Marshal în spațiul de nume `System.Runtime.InteropServices`:

```
using System.Runtime.InteropServices;
Marshal.ReleaseComObject(myAuthorRCW);
```

6. Aplicatii Laborator:

- 1.Implementati programul de mai sus, folositi-va de sursele atasate acestui laborator.
- 2.Adaugati inca doua metode de cautare pe langa titlu, si autor, de exemplu gen si dimensiune(numar pagini).

Bibliografie :

1. <https://www.codeproject.com/Articles/9776/COM-Interoperability-in-NET-Framework-Part-I>
2. <http://www.4guysfromrolla.com/articles/010803-1.aspx#postadlink>